把主线收窄，把能力做实

为张子健制定 · 2026 年 10 月底启动，约至 2028 年 4 月底 · 每周 12-15 小时

Swift / iOS 主修 + TypeScript / React Native 跨端
Kotlin / Android 辅修 + 最小 Go 后端

你已有 4 个月字节国际支付 iOS 实习经历。训练重点是把 AI 辅助完成需求，逐步变成自己能分析、实现、调试、验证的能力。保留 MeetLoop 活动组织与报名产品，用同一业务反复练习。

18 个月后的目标

争取达到移动端实习 / 初级岗位候选人的能力要求：iOS 能独立承担边界清楚的需求；RN 能交付一个跨端模块；Android 能完成小链路并排查宿主问题；Go 能支撑业务并保证关键数据正确。实际结果取决于练习质量、反馈与招聘条件。

主次	练到什么程度
iOS：主项	先 Swift / SwiftUI，再补 UIKit 专项；深入状态、网络、并发、生命周期、内存、测试与性能。
RN：第二能力	第 6-9 月：JS / TS / React 基础，独立报名模块，再嵌入 iOS。
Android：辅项	第 10-11 月：Kotlin / Compose 列表与详情，理解生命周期、协程和 RN 宿主。
Go：支撑能力	一个服务、一个数据库；保留权限、容量、报名事务与幂等，按 iOS 实际需要增加接口。

现在与 Mac 到货第一晚

• 现在：从 100 Days of SwiftUI 的 Day 0-1 开始，在在线 Swift 环境写变量与打印练习；具体任务见第 14 页。

• 到货第一晚：安装兼容的正式版 Xcode，创建 SwiftUI App 并运行模拟器；语法与状态基础通过后再做活动列表。

v2.2（2026-10-19）：统一 100 Days 入门主课，UIKit 后续专项；iOS 基础延长至第 5 月，RN / Android 顺延；首版只做报名小链路，第 2-3 月开始原型反馈。第 4 页看总安排，第 9、14-16 页看开始方式和课程。文件名保留首次创建日期。

就业判断：有机会，要用证据争取

事实与推断分开 · 检索截至 2026-10-16 · 样本不是岗位总量统计

当前可以确认什么

TikTok 在 San Jose 的 2027 Summer 移动端实习岗位，列出基础编程与数据结构要求，接受 Swift、Kotlin 等语言；移动端渲染、线程、内存、生命周期和性能工具属于相关能力。[1] 这支持“深挖客户端基础”的方向，但不代表你已满足其全部资格。

Amazon 的 2026 US 前端岗位要求 HTML、CSS、JavaScript 和计算机基础。[2] 因此，你这条路线更适合定位 Mobile / iOS / React Native。若以后投 Web 前端，还要专门补浏览器、DOM、CSS 和 Web 工程能力。

BLS 当前预测美国软件开发、QA 与测试相关职业整体在 2025-2035 年增长 10%。[3] 这是宽口径、长期的职业预测，不能换算为 2028 年移动端校招名额、国际学生机会或个人录用概率。

对两年后与 AI 的判断

无法可靠保证两年后好找工作，也没有证据支持“大前端届时会全部消失”。我的判断是：常规页面和样板代码越来越容易生成，独立判断需求、定位跨端故障、验证质量、解释取舍的能力会更值得投入。这是职业策略判断，不是对未来岗位数量的预测。

不把“学会更多框架”当成竞争力。把“独立交付过什么，遇到什么问题，如何证明修好了”变成竞争力。

求职从现在并行，而不是第 18 个月才开始

- 现在整理实习中真实负责的需求、解耦边界和验证方式；不使用公司私有代码，不编造个人贡献或指标。
- 每季度抽样阅读 15-20 个与你毕业时间、地点和身份条件匹配的岗位，记录必需项与加分项，调整后 3 个月。
- 第 3-6 个月准备第一版简历和演示，按开放时间申请符合资格的机会；美国学校的项目时间与申请资格单独核实。

[1] TikTok：2027 Summer 移动端实习，San Jose

[2] Amazon：Front-End Engineer 2026（US）

[3] BLS：Software Developers 职业展望

先做报名小链路，再扩展跨端

首轮：活动列表 → 活动详情 → 报名 / 取消 → 我的报名；活动先用预置数据

部分	业务职责	学习价值
iOS / Swift	主客户端先实现四步小链路；创建 / 编辑活动和组织者管理在基础验收后再补。	SwiftUI 起步；后续用 UIKit 做一处列表 / 详情专项，复用同一数据层。
共享 RN / TS	第 6-9 月实现确认表单 → 提交 → 结果；先独立运行，再替换 iOS 的报名入口。	React 状态、异步请求、组件、宿主交互与原生模块。
Android / Kotlin	第 10-11 月：原生列表与详情，加同一个 RN 报名入口。	Compose、生命周期、协程、构建与调试；不追求功能对齐。
Go + MySQL	由 Mock 过渡到活动读取、会话 / 权限、报名和取消；活动写入接口按需补。	HTTP、SQL、事务、错误契约与运维基础。

按阶段形成边界

最初：SwiftUI 页面 → 简单状态 / Repository → 本地数据。之后换成真实 API；同一层数据支持 UIKit 专项。到跨端阶段再形成原生宿主 → RN 报名模块 → HTTPS / JSON → Go 单体 → MySQL。不要第 1 月预建全部目录和适配层。

RN 接入时写清输入与输出：活动 ID、会话 / 请求接口；完成、取消、需登录、待确认。明确谁负责导航、任务取消和原生页面刷新。

后端弱化规模，保留正确性

• 保留鉴权与权限、参数校验、统一错误、迁移、数据库约束、报名事务与幂等。客户端按钮置灰不能代替服务端防重。

• 首版报名只包含已确认、已取消；同一用户与活动一条主记录，取消后暂不开放重报。超时先查状态；创建 / 编辑活动后置。

• 候补、推送、聊天、推荐、真实资金支付、RPC、Kafka、Kubernetes 均非必做。只维护一个服务和数据库。

仓库按需增加 apps/ios、apps/android、packages/registration-rn、server、docs。共享纯 TS 逻辑，平台能力放在适配层；接入跟随所选 RN 版本的官方说明。[10]

18 个月路线与每周节奏

从 2026 年 10 月底起算；阶段验收决定是否切换技术栈，月份是排期参考

阶段	主要任务	进入下一阶段的证据
第 1-5 月	Swift / SwiftUI 与报名小链路；再补 UIKit、网络、调试和最小 Go API。	独立改需求；处理四态与取消；关键规则有验证；能断点定位并解释数据流。
第 6-9 月	JS → TS → React → RN 报名模块；先独立运行，再接入 iOS。	解释状态和副作用；处理失败与待确认；返回原生页后状态一致。
第 10-11 月	Kotlin / Compose 小链路；相同 RN 模块接入 Android。	生命周期合理；能跨层排错；两端 Release 构建可运行。
第 12-15 月	iOS 深化；弱网、缓存、性能、测试、持续用户反馈；最多一个专题。	故障能复现和回归；有实测报告与两轮修复；形成 3 个案例。
第 16-18 月	集中面试、投递、项目讲解与协作；修补薄弱项。	能现场解释和修改；作品可复现；实际资格与岗位匹配。

每周 12 小时保底；课程包含在内

8 小时学习与实现：起步时约 4 小时课程跟做、4 小时独立练习 / 项目；基础建立后改为 2 小时文档、6 小时项目。**2 小时基础 / 算法**；**1 小时英语**；**1 小时复盘 / 岗位研究**。额外 3 小时优先补调试与薄弱项。

可分成 4 个晚上各 1.5 小时，周末合计 6 小时。18 个月约 78 周，共 936-1170 小时；预留 20% 给考试、搬家和返工后，约有 750-940 小时。时间估算不等于能力承诺。

阶段切换与取舍

• 每 4 周演示一次。iOS 验收不过就继续缩小范围和补练，不因到第 6 月而强行启动 RN。课程进度不必与日历日一一对应。

• 第 2-3 月开始让 2-3 人体验原型；简历和岗位研究从现在并行，按实际招聘窗口投递，不等第 16 月。

• 延期先删 Lynx、额外沙盒业务和 Android 次要页面。跨端阶段仍给 iOS 留维护时间；接单试验也必须计入总预算。

第 1-5 月：建立 iOS 独立交付能力

主线：100 Days of SwiftUI → MeetLoop 小链路 → UIKit 专项与最小 API

时间	具体任务	必须理解 / 验收
第 1 月	Swift 基础与课程练习；再用 SwiftUI 显示预置活动列表和详情。	变量、集合、可选值、结构体、闭包；能解释数据如何驱动页面。
第 2-3 月	报名 / 取消与我的报名；逐步学习网络四态、Codable、持久化；让 2-3 人体验原型。	状态来源、错误传播、任务取消、MainActor；根据观察修复至少一个流程问题。
第 4 月	用 Today 补 UIKit 列表 / 详情，复用数据层；练布局、复用、生命周期和内存调试。	能解释 SwiftUI / UIKit 的职责差异；用断点和 Memory Graph 定位问题。
第 5 月	接最小 Go + MySQL；会话 / 权限、活动读取、报名与取消；补关键测试。	HTTP 契约、令牌存储、事务、约束与幂等；创建 / 编辑活动有余力再补。

在同一业务中做四个实验

- 快速切换活动，让旧请求晚到；确保旧结果不覆盖当前页面，并说明任务取消的责任。
- UIKit 列表快速滚动时不出现图片错位；解释复用、取消和缓存各自的作用。
- 反复进出详情页，定位并修复一次人为引入的循环引用；保留复现和验证记录。
- 模拟空数据、坏 JSON、错误码、超时与重复报名；为关键分支写可重复验证的用例。

第一轮架构与反馈

按功能组织 View、简单状态 / ViewModel、Repository；需要切换 Mock 与真实 API 时再引入协议。早期试用可在你设备上操作，不要求先部署服务器。记录用户做了什么、卡在哪里，再改流程。

进入 RN 前：不让 AI 整段生成，独立完成筛选并保留状态；解释四态和一次超时；用断点定位问题，演示关键规则验证。课程未吸收就延长本阶段。

100 Days 按吸收情况继续，不能用刷完视频代替独立练习；UIKit 是随后专项。后端可用 AI 搭骨架，但实际调用的接口、SQL、权限与测试必须读懂。

[18] Paul Hudson：100 Days of SwiftUI

[5] Apple：UIKit Today 教程

[17] Go：官方交互式入门

第 6-9 月：补 JS，再进入 RN

前提：iOS 小链路已通过独立验收；四个月约 16 周，工具安装与返工也计时

顺序	学习内容	小型输出
JS / TS 约 4 周	作用域、闭包、集合、模块、Promise、async / await、事件循环；联合类型、收窄和泛型基础。	纯 TS 报名模型与校验函数；解释异步执行顺序。
React 约 3 周	组件、props、state、事件、表单、effect 与清理；基本 HTML / CSS / Flexbox。	小型 Web 报名表单；理解状态驱动 UI，不扩成第二个产品。
RN 约 5 周	组件、布局、列表、键盘、导航、请求、错误与平台差异。	独立运行的报名确认 / 提交 / 结果模块，用 Mock 后接 API。
iOS 集成 约 4 周	原生入口、导航、会话、关闭与刷新契约；按官方说明构建。	嵌入现有 iOS App；Release 不依赖 Metro，返回原生页状态正确。

必须能回答的问题

- 什么变化导致重新渲染？为什么不能随意在渲染中发请求？effect 何时执行与清理？
- 提交中、失败、结果待确认有何区别？如何避免过期请求、旧闭包和重复提交？
- 哪些是 React 通用能力，哪些属于 RN 平台 API？DOM / CSS 为什么不能原样搬到 RN？
- 状态应放在哪里？先用本地 state；出现实际共享需求后再选择状态管理工具。

第 9 月先完成一个宿主

固定输入为活动 ID、初始信息、会话 / 请求适配接口；输出为完成、取消、需登录、待确认。RN 不控制整个 App 导航。先让 iOS 返回、重进、登录失效和刷新正确，第 10-11 月才补 Android。

阶段验收：独立修改报名规则，解释从输入到渲染和提交的数据流；排查一次副作用错误与一次原生刷新问题；保留演示和回归用例。

课程与集成都包含在每周预算内；锁定工具链和依赖版本。余力不足时缩减 RN 页面，保住状态正确性与 iOS 维护时间。

[6] MDN：Web 开发基础

[7] TypeScript：Handbook

[8] React：Learn

[9] React Native：入门

[10] React Native：集成到已有原生应用

第 10-11 月：Android 辅修与集成

目标：理解 Kotlin / Android 原生宿主，能维护一个小链路；不追求三端功能对齐

Kotlin / Android 学习边界

先学 Kotlin 空安全、数据类、集合和 lambda；再选学 Compose 状态、Activity / ViewModel 生命周期、协程、Flow、网络与 Gradle。官方 Compose 课程只学当前链路所需内容。[11]

月份	产出	验收
第 10 月	Kotlin 小练习与原生列表 → 详情；调用已有活动 API。	旋转 / 重建后状态合理；请求受生命周期约束；会看日志和断点。
第 11 月	把已在 iOS 跑通的 RN 报名模块嵌入 Android；完成一个平台能力适配。	两端进入、关闭、报名、重开一致；Release 构建不依赖 Metro。

只选一项原生能力练习

优先选择系统分享；也可选择安全会话存储或文件选择。定义 TS 接口，并在 Swift / Kotlin 侧适配；按所选能力处理取消、错误、宿主销毁和必要权限。使用现成库时，读懂它与业务之间的边界。

学习所选 RN 版本的原生组件 / 模块机制，概述 Fabric、TurboModules、JSI 的职责；暂不要求通读源码或写 C++ 引擎。接入跟随官方文档。[10]

至少做一次完整跨端排错

- RN 显示报名成功，而原生详情仍显示未报名：沿请求、回调、状态刷新逐步检查，留下复现与修复记录。
- 覆盖前后台切换、Android 返回键、重复打开、会话失效与重复回调；不要只验证成功路径。

验收：能沿用户点击追到 Kotlin / Swift 宿主、RN、Go API，再解释结果如何返回。Android 只要求这个小链路；不达标时减少页面或压缩后续可选专题。

持续维护主项

这两个月每周仍给 iOS 留一次修改或调试，计入每周 8 小时学习与实现。iOS 接入已在第 9 月完成；此阶段以 Android 基础与跨端差异为重点。

[10] React Native：集成到已有原生应用

[11] Android：Android Basics with Compose

第 12-15 月：把作品变成工程证据

第 2-3 月已开始原型反馈；本阶段集中做可靠性、实测与完整跨端试用

第 12-13 月：稳定、测量、反馈

• 可靠性：弱网、超时与结果待确认、重复提交、会话失效、缓存更新。服务端始终是报名结果的权威。

• 性能：选一个 iOS 滚动 / 图片 / 内存问题和一个 RN 渲染或集成问题。固定设备、构建、数据量与场景，实测改前改后，不预设提升百分比。

• 测试与 CI：延续早期的业务分支测试，补 API 集成测试与一条核心界面回归；CI 执行能自动运行的关键检查。

• 完整试用：Go 服务具备 HTTPS、日志、迁移与备份恢复说明；让 3-5 位同学试用，完成两轮修复。保留第 2-3 月起的反馈记录。

第 14-15 月：一个专题，先深化已有业务

优先深化报名状态、异步回调、重复提交、取消与结果确认，并解释页面与业务边界。若主线已稳，再选择“沙盒收银台模块解耦”替代该专题，延续你的支付经历；使用模拟渠道或沙盒。

Objective-C / Swift 互操作、复杂列表或内存性能按目标岗位择一补充，计入专题时间。留下问题、边界、取舍、验证与回归风险；不要同时增加多条业务线。

RN → Lynx：按岗位需要替换可选时间

如实际目标团队使用 Lynx，可用 2-3 周迁移已有报名模块：复用 React / TS 概念和纯逻辑，重新处理组件、样式、事件、线程与原生接口。React 风格不等于 RN 代码直接运行。[12][13]

记录能复用与必须重写的部分。Lynx 从专题阶段扣时间，主线未验收就取消；Flutter、KMP 不加入必修。明确岗位机会可以触发顺序调整。

最终留下 3 个可讲案例：功能设计与取舍、复杂故障定位、可复现的性能或可靠性改进。每个都有代码、演示或验证证据。

[12] Lynx：ReactLynx 简介

[13] Lynx：生命周期与线程相关语义

Mac 到货后：第一轮小链路任务

按顺序推进，约两周项目时间；若 Swift / SwiftUI 状态基础尚未通过，就拆小并顺延

任务	用时参考	完成标准
第 1 次	1.5 小时	兼容的正式版 Xcode、一个模拟器、Git；SwiftUI 空白工程运行并首次提交。
第 2 次	1.5 小时	把已理解的 Event 与 3 条数据放入 SwiftUI List；不熟悉结构体时先补对应课。
第 3 次	1.5 小时	点击活动进入详情并返回；解释导航和数据传递。
第 4 次	1.5 小时	用简单本地状态实现报名 / 取消，显示“我的报名”；理解状态来源。
第 5 次	1.5 小时	连续点击、返回和重进，检查重复占名额与状态不同步；练断点。
第 6 次	1.5 小时	增加可切换的加载、空、错、成功状态；不要求此时理解全部异步 API。
第 7 次	1.5 小时	独立增加“已结束不可报名”规则；检查满员、重复、取消等边界。
第 8 次	1.5 小时	录制 3 分钟演示并解释数据流；整理未理解项和下一步计划。

以上共约 12 小时实现任务。起步每周约 4 小时课程、4 小时独立练习 / 项目，可能需要三周或更久。另留 2 小时基础、1 小时英语、1 小时复盘；不为赶两周而跳过语言基础。

到货第一晚只要求工程跑起来

环境安装耗尽 90 分钟时，以空白 App 可运行为完成标准。之后完成 100 Days 当前小节，再把学会的能力迁移到项目。Today / UIKit 留到后续专项。

第一轮只回答这四个问题

- 数据存在哪里，点击和状态变化经过哪些函数?
- 为什么页面更新，为什么会产生重复报名或状态不同步?
- 能否在断点查看变量，并预测下一步结果?
- 换一条报名规则，自己会改哪里、验证什么?

Mock Repository 的延迟模拟、真实网络与数据库在基础稳定后再加入。第一轮只要求小链路可运行，并能独立解释与修改。

怎样把 AI 辅助变成自己的能力

AI 可以继续用；每周必须留下独立推理、调试与修改的证据

每个需求固定做 5 步

- 先自己写：输入、输出、状态、失败情形和 3 条验收条件。用几句话就够。
- 先实现一个最小版本。卡住时记录现象与猜测，再问 AI 原理、排查方向或小段示例。
- 看生成代码的 diff：能解释依赖、异步边界、错误处理；无法说明的部分要缩小或重写。
- 用断点、日志、测试和真实运行验证。AI 说“没问题”不算验收。
- 隔一天重新讲解，并改一个条件。每周保留一次 45-60 分钟不用代码生成器的小练习，可查文档。

基础课不另开三条重型战线

基础	与项目相连的练法
数据结构与算法	数组、哈希、栈队列、树、二分、递归、常见图遍历；能讲复杂度并写边界测试。先固定一种熟悉的面试语言。
系统与网络	进程 / 线程、竞态、内存、HTTP / HTTPS、DNS、超时与重试；结合并发请求与弱网问题理解。
数据库与工程	索引、事务、约束、幂等；Git、调试、测试、CI；只学当前业务中能举例说明的部分。
沟通与英语	每周用英语讲 5 分钟改动；逐步做到 15 分钟问答，能解释 trade-off、bug 与验证。

第 16-18 月的求职材料与验收

项目仓库有可复现运行说明、短演示、架构图、关键用例与 3 个案例。至少经历一次外部代码评审、团队课程项目、开源协作或后续实习，把接受反馈与协作过程纳入证据。

做 45-60 分钟模拟面试：澄清需求、编码、测试、解释复杂度；再演示项目并现场修改。以暴露的问题决定下一周练什么，不以刷题数量或框架数评价自己。

理想结果：你能对自己提交的代码负责，面对陌生的小问题有排查方法。这比“我用过 Swift、RN、Kotlin、Go”更接近岗位需要。

一台 Mac 能承担哪些任务

按你计划的 Apple Silicon Mac、32GB 内存、1TB 存储评估

足够承担本计划的主要开发与本地测试。无需为了学习再配一台电脑；后期真机测试与在线服务需要额外设备或服务条件。

Android Studio 官方推荐配置包含 32GB 内存与 Apple Silicon。[15] Apple 已有 M6 官方信息。[16] 对同时运行 Xcode、一个或两个模拟器、编辑器及轻量 Go / MySQL 的判断，是针对本计划规模的工程估计，不是所有工程的性能保证。

阶段	安装什么	检查什么
开始时	正式版 Xcode 27、一个 iOS 模拟器、Git。	macOS Tahoe 26.6 或更新版本；能构建运行、断点和提交。详见第 13 页。
接服务端时	Go；MySQL；需要容器时再装兼容 Apple Silicon 的容器工具。	优先 ARM64 镜像；端口与数据目录明确；可运行 API 和迁移。
RN 阶段	按所选 RN 版本准备 Node、包管理器及 iOS 依赖。	保存锁文件；版本写入 README；模板与原生集成文档一致。
Android 阶段	Android Studio、匹配的 JDK / Gradle、一个 ARM Android 模拟器。	Debug 与 Release 都能运行；宿主与 RN 版本兼容。

资源管理

• 常用时开一个模拟器，做跨端对照时再开两个。容器只运行数据库和当前服务；不需要本地大模型。

• 按经验建议保留约 150-200GB 空闲空间，定期清理不用的模拟器和旧构建缓存；这是维护建议，并非官方最低要求。

• 1TB 不等于备份。源码使用 Git 与远程仓库，数据库备份和必要本地资料另行保存。

Mac 之外的边界

模拟器不能覆盖所有相机、权限、推送、发热、电量与真机性能问题。后期借用或使用一台 iPhone 和一台 Android 真机做抽样验证即可。需要朋友随时访问时，后端需在线部署；商店分发、云资源与账号条件到发布阶段再核实。

[14] Apple：Xcode 系统要求

[15] Android Studio：安装与系统要求

[16] Apple：M6 官方发布信息

资源、原计划变更与复盘表

入口汇总见本页；每门课学哪些内容、何时学，详见第 15-16 页

官方学习入口

[4] Apple：Swift 官方语言指南

[5] Apple：UIKit Today 教程

[6] MDN：Web 开发基础

[7] TypeScript：Handbook

[8] React：Learn

[9] React Native：入门

[10] React Native：集成到已有原生应用

[11] Android：Android Basics with Compose

[12] Lynx：ReactLynx 简介

[13] Lynx：生命周期与线程相关语义

[17] Go：官方交互式入门

v2.2 修订记录（2026-10-19）

沿用 v2.1 的 iOS 主项、RN 共享报名模块、Android 辅项和最小 Go；修正课程与首两周任务：100 Days / SwiftUI 主课在前，Today / UIKit 专项在后。Swift 官方指南作参考，CS193p 仍为可选。

iOS 调整为第 1-5 月，RN 第 6-9 月，Android 第 10-11 月，深化第 12-15 月。首版使用预置活动，只做列表、详情、报名 / 取消、我的报名；创建 / 编辑后置。第 2-3 月加入原型反馈；按验收切阶段，课程、基础与可选接单都计入每周总预算。

每月只填写这 6 行

记录项	填写提示
本月完成	可运行的功能或可复现的问题修复是什么?
独立掌握	不靠整段生成，自己能解释并修改哪一部分?
技术证据	提交、测试、测量、录屏、复盘分别在哪里?
仍不理解	最多列 3 个具体问题，避免写“基础不好”。
外部反馈	代码评审、试用、面试或岗位要求给了什么反馈?
下月取舍	最重要的一个里程碑；明确延期或删除什么。

来源说明：岗位与工具信息沿用 2026-10-16 核对记录，并非本次重新统计。课程和 RN 接入入口在 2026-10-19 复查；动态内容以使用时官方页面为准。训练安排与时长为个体建议，岗位样本不代表市场总量。

Xcode 27：版本与教程怎么配合

可使用 Xcode 27；建议从正式版开始，在每个阶段记录实际工具版本

Apple 当前列出的 Xcode 27 系统要求：macOS Tahoe 26.6 或更新版本。[14] 到货后检查系统版本，再安装正式版 Xcode 27 系列。

先区分三个版本概念

概念	含义	本项目的处理方式
Xcode 版本	开发工具与编译器、SDK 的组合。	用正式版 Xcode 27 系列；阶段内不为追新反复升级。
SDK / 模拟器	编译时可用的系统接口，以及模拟运行的系统。	先装一个兼容的 iOS 模拟器；不需要下载所有平台组件。
最低系统版本	你的 App 允许运行的最低 iOS 版本。	它不等于 Xcode 版本；选定依赖后确定，并写入 README。

后续 UIKit 专项：Today 的版本差异

第 4 月前后再用 Today 学 UIKit 控制器、collection view、diffable data source、导航和编辑。部分项目文件与截图标注 Xcode 16，按当前工具找到相同功能即可；遇到废弃 API 或编译错误时查看当前说明，不需要为了复刻截图退回旧工具。[5][25]

• 教程使用 Storyboard。先按教程跑通；若模板入口不同，可从官方 starter project 开始。后续再把一个小页面改成纯代码布局作为练习。

• Swift 并发诊断与项目设置可能和旧示例不同。记录 Swift Language Mode 与 Default Actor Isolation；理解错误涉及的 actor / 数据边界，再做最小修改。

• 界面截图不一致不代表概念失效。把差异记录成“原写法、当前写法、为什么”，不要一次让 AI 重写整个示例。

新工程第一次运行的验收

一个 SwiftUI 页面能够运行；断点能停住并查看变量；Git 有首次提交；README 记下 macOS、Xcode、模拟器系统版本。做到这些，再把第 14 页的活动数据移入工程。

现在的 Windows 可以练纯 Swift 语言和命令行逻辑；Apple 的 UIKit / SwiftUI 界面、iOS 模拟器与签名调试留到 Mac / Xcode 阶段。

[14] Apple：Xcode 系统要求

[25] Apple：App Dev Tutorials 总目录

Mac 到货前：从 Day 0-1 开始

2026 年 10 月 19 日修订 · Windows + 浏览器即可 · 当前只启动一门主课

今天 90 分钟：10 分钟读 Day 0 → 30 分钟学 Day 1 → 35 分钟独立写变量与打印 → 15 分钟解释和复盘。[18][26]

打开 SwiftFiddle，先写活动名称 String 和剩余名额 Int，打印一句活动介绍，再修改数值观察结果。Day 2 再加入是否开放 Bool；还没学到结构体和闭包时，不强行完成 Event 模型或筛选功能。

按课程顺序	练习与迁移	过关标准
Day 1-6 基础与控制流	常量、变量、集合、条件、循环；逐步保存多个活动，并筛选可报名项。	能解释类型；独立改条件；空数据和边界有结果。
Day 7-11 函数与结构体	把查询和校验提取成函数；学到 struct 后再建立 Event，加入 ID、时间与容量。	能解释参数、返回值和实例；用闭包筛选排序。
Day 12-15 类、协议、可选值与复习	按 ID 查活动；用可选值处理不存在；比较值与引用，完成课程检查点。	不用强制解包掩盖错误；能脱离示例修改。
基础通过后 纯 Swift 小练习	用枚举描述结果；实现内存报名 / 取消；模拟满员、重复报名、已结束。	不重复占名额；取消恢复容量；首版取消后不重报。
任意一个复盘时段	画列表 → 详情 → 报名 → 我的报名草图，写 5 条用户故事。	有草图、已懂 / 未懂清单和保存的代码。

Mac 到货不意味着必须跳过基础

课程的 Day 是学习单元；卡住可用多个学习时段。到货前不要求刷完 Day 15。到货后先验证环境，再继续当前课程；语言与状态基础够用时按第 9 页迁移到 App。

每周 8 小时分给课程与独立练习（起步各约 4 小时），另留 2 小时基础 / 算法、1 小时英语、1 小时复盘。今天代码能运行后，关掉教程重写一次，解释每个变量为什么这样定义。

在线环境只练纯 Swift，不提供 iOS UI 预览；不可用时可按 Swift.org 安装 Windows Swift。[27] 服务端防重与事务以后再补；现在不启动 RN、Android、Today 或 CS193p 的完整课程。

[18] Paul Hudson：100 Days of SwiftUI

[26] SwiftFiddle：在线 Swift 运行环境

[27] Swift.org：Windows 安装说明

iOS 课程：一门主课，学完就用

100 Days of SwiftUI 为当前主课；Today / UIKit 随后专项，CS193p 可选后续深化

资料与入口	何时与怎样使用	MeetLoop 中的练习
100 Days of SwiftUI	先 Day 1-15 语言与复习，再顺序推进 SwiftUI、项目和挑战；按吸收程度延长，不按 100 个日历日催进度。	列表、详情、状态、报名规则和数据处理；课程学到相关点就迁移。
Swift 官方语言指南	作为参考书，遇到可选值、协议、ARC、错误与并发问题时查对应章节。	写最小例子验证理解，再回到当前功能。
Apple Today / UIKit	第 4 月前后、SwiftUI 小链路通过后，选学前四章；UIKit 专项，不另开第二条长期主课。	复用已有数据层，做一处 UIKit 列表 / 详情，再按需补编辑。
Apple Develop in Swift	官方补充参考；只在主课对应概念或 API 有卡点时查，不同时完整跟做。	核对布局、状态、输入和建模写法。
Apple Earthquakes	网络阶段选学 JSON、依赖替换、并发、缓存与测试，计入当周学习时间。	接活动 API，处理坏 JSON、失败、取消与关键用例。

主课与项目怎样配合

每完成一个课程项目或阶段挑战，关掉教程，在 MeetLoop 迁移一项已经学会的能力，再改一个条件。与活动业务无关的地图、图像等内容作为课程练习，不强行塞进产品。

UIKit 专项保留，顺序后移

Today 第 1-2 章用于列表、导航和控制器；第 3-4 章用于编辑、增删和筛选。先做一处完整模块，理解布局、复用、生命周期，以及与 SwiftUI 的互操作；不重写整个 App。

课程完成标准：能自己解释、实现、修改和验证。100 Days 不是就业能力的全部；UIKit、调试、网络、测试和项目反馈按阶段继续补。

Paul Hudson 的 100 Days 为第三方课程；Apple 文档负责核对官方 API。每周课程与实作合计约 8 小时，完整课程所需日历时间取决于练习和返工，不另加预算。

[18] Paul Hudson：100 Days of SwiftUI

[25] Apple：App Dev Tutorials 总目录

进阶与跨端课程：按阶段打开

可点击表中链接 · 延续每周 12-15 小时总预算 · 同一阶段保留一条主学习线

何时使用	资料	学到什么就转回项目
主课与小链路基础扎实后	Stanford CS193p Spring 2025	视频配作业，深化模型、状态和类型系统；100 Days 与小链路基础扎实后再选学，替换部分学习时间，不与主课全量并行。
网络与并发阶段	Embracing Swift concurrency Code-along 实操	理解任务、actor 隔离和数据竞争；用请求取消与图片处理练习。新版设置以所用工具文档为准。
性能阶段	SwiftUI performance with Instruments	观察视图更新和耗时，复现一个真实问题并记录改前改后。
第 6 月起 JS / CSS	MDN Learn Web Development	JS 函数、对象、闭包、异步；HTML、盒模型、Flexbox。输出一个简单报名表单。
JS 基础之后	TypeScript Handbook	常用类型、联合类型、类型收窄、函数、泛型基础；写报名数据与结果类型。
第 6-9 月 React / RN	React Learn React Native 入门 接入已有原生应用	先组件、交互、状态与副作用，再做 RN 模块；第 9 月接入 iOS，第 10-11 月补 Android。
第 10-11 月 Android	Android Basics with Compose	按需学习 Kotlin、Compose、状态、导航、网络与持久化；完成原生小链路和 RN 宿主。
接 Go API 时	A Tour of Go	语言基础后围绕实际接口补 HTTP、SQL 和测试；不延伸成独立的后端大项目。
明确需要 Lynx 时	ReactLynx 简介 生命周期与线程	迁移一个已完成的 RN 模块，记录复用与重写部分；主线未通过则后置。

怎样判断需要更多课程

先写清当前卡点：语法、状态、网络、调试，还是算法。当前资料解决不了，再补对应章节或短课。数据结构与算法继续使用已有课程和题目复盘，每周 2 小时纳入总预算。

完成标准：自己能解释、实现、修改并验证。每次只选一个与本周任务直接相关的章节；读完马上回到项目。